5 Έλεγχος Ροής

Ο έλεγχος της ροής του προγράμματος πραγματοποιείται με τις δομές επιλογής και τις επαναληπτικές δομές. Οι δομές επιλογής μας δίνουν τη δυνατότητα να δημιουργούμε πολλαπλές διαδρομές μέσα στον κώδικά μας και να επιλέγουμε κάθε φορά την καταλληλότερη ανάλογα με τις συνθήκες που επικρατούν κατά την εκτέλεση του προγράμματος. Με τις επαναληπτικές δομές ρυθμίζουμε την επανάληψη τμημάτων του κώδικά μας, επίσης, ανάλογα με συγκεκριμένες συνθήκες που επικρατούν κατά την εκτέλεση του προγράμματος. Τόσο οι δομές επιλογής όσο και οι επαναληπτικές δομές αποτελούν βασικές δομές προγραμματισμού, όχι μόνο στην Java αλλά σε κάθε προγραμματιστικό περιβάλλον.
Επιπλέον, τόσο οι δομές επιλογής όσο και οι επαναληπτικές δομές βασίζονται στον έλεγχο μιας συνθήκης (condition). Η συνθήκη είναι μία λογική έκφραση, δηλαδή μία έκφραση τύπου boolean. Για παράδειγμα, η έκφραση i>5 && j<0 είναι λογική έκφραση, καθώς λαμβάνει την τιμή true ή false ανάλογα με την τιμή των i και j.

5.1 Δομές επιλογής

Οι δομές επιλογής μας δίνουν τη δυνατότητα να καθορίζουμε διαφορετικές διαδρομές που ακολουθεί ο κώδικάς μας ανάλογα με τις συνθήκες που επικρατούν στο πρόγραμμά μας κατά τον χρόνο εκτέλεσης. Πρόκειται για θεμελιώδεις δομές που συναντώνται με τη μία ή την άλλη μορφή σε κάθε γλώσσα προγραμματισμού. Για να αντιληφθεί κανείς τη σπουδαιότητά τους αρκεί να αναλογιστεί πως χωρίς αυτές τα προγράμματα θα ήταν μια συλλογή εντολών που θα εκτελούνταν αναγκαστικά σειριακά. Με άλλα λόγια, η λήψη αποφάσεων κατά την εκτέλεση του προγράμματος θα ήταν αδύνατη. Για παράδειγμα, αν ένα πρόγραμμα διέθετε ως δεδομένα μια σειρά από ονόματα και κάθε όνομα συνδεόταν με χαρακτηρισμό φύλου, δεν θα ήταν σε θέση να προσφωνήσει τους άντρες ως κυρίους και τις γυναίκες ως κυρίες.

Η Java υποστηρίζει τις δομές επιλογής if, if…else, switch και τον τριαδικό τελεστή. Είναι όμως εφικτό να συνδυαστούν οι δομές if και if…else διαμορφώνοντας έτσι μια ιδιαίτερη δομή γνωστή ως εμφωλευμένη (nested) if. Στη συνέχεια παρουσιάζονται μια προς μια οι δομές επιλογής της Java.

5.1.1 Η επιλογή if

Η γενική μορφή της if δίνεται ακολούθως:

```
if (condition) {
     statement(s);
}
```

Η δεσμευμένη λέξη if ακολουθείται από τη συνθήκη ελέγχου μέσα σε παρενθέσεις. Αν η συνθήκη είναι αληθής, η ροή του προγράμματος μπαίνει στο μπλοκ που ακολουθεί και εκτελεί τα statements. Αν η συνθήκη είναι ψευδής, η ροή του προγράμματος μεταπηδά στην έξοδο από το μπλοκ της if.

Μέσα στο μπλοκ της if μπορούμε να έχουμε ένα ή περισσότερα statements. Αν έχουμε μόνο ένα, οι αγκύλες είναι προαιρετικές, ωστόσο, συνηθίζεται να τοποθετούνται.

```
static void simpleIf() { //Κώδικας 5.1
    int i = 1, j = -1;
    if (i > 0 && j < 1) {
        System.out.println("i μεγαλύτερο από το 0");
        System.out.println("j μικρότερο από το 1");
    }
    System.out.println("Μετά τον έλεγχο της έκφρασης \"i>0 && j<1\"");
}
```

Κώδικας 5.1 *Παράδειγμα απλής if*

Στον κώδικα 5.1 η τιμή της συνθήκης ελέγχου εξαρτάται από τις τιμές των i και j. Αν για παράδειγμα, i==1 και j==-1, η συνθήκη είναι αληθής, οπότε οι προτάσεις "i μεγαλύτερο από το 0" και "j μικρότερο από το 1" θα εμφανιστούν. Αν όμως το i είναι μικρότερο ή ίσο με το 0 ή το j είναι μεγαλύτερο ή ίσο με το 1, η συνθήκη είναι ψευδής, οπότε αυτές οι προτάσεις δεν θα εμφανιστούν. Σε κάθε περίπτωση, θα τυπωθεί η πρόταση "Μετά τον έλεγχο της έκφρασης "i>0 && j<1"".

```
static void IfWithSimpleStatement() { //Κώδικας 5.2
     int i = 0;
     if (i > 0) {
         System.out.println("Το i είναι μεγαλύτερο από το 0");
     }
     System.out.println("Μετά τον έλεγχο της έκφρασης \"i>0\"");
}
```

Κώδικας 5.2 *Παράδειγμα απλής if με ένα statement*

Ο κώδικας 5.2 παρουσιάζει μια if που εφόσον η συνθήκη της αληθεύει εκτελεί μια μόνο πρόταση. Σε αυτήν την περίπτωση δεν είναι υποχρεωτικό να χρησιμοποιηθούν αγκύλες. Ωστόσο, στην πράξη, συχνά τοποθετούνται αγκύλες από πολλούς επαγγελματίες αλλά και από πολλά IDE, ακόμη και στην περίπτωση μίας πρότασης.
Η πρόταση που ακολουθεί μια if, μπορεί να είναι και η ίδια μια if. Έτσι προκύπτει μια δομή γνωστή ως εμφωλευμένη if (nested if).

```
static void nestedIf() { //Κώδικας 5.3
     int i = 1, j = -1;
     if (i > 0) {
         if (j < 1) {
             System.out.println("i>0 και j<1");
         }
     }
}
```

Κώδικας 5.3 *Παράδειγμα εμφωλευμένης if*

Ο κώδικας 5.3 ελέγχει καταρχάς αν το i είναι μεγαλύτερο του μηδενός. Αν η συνθήκη αληθεύει, προχωρά στην εκτέλεση της επόμενης πρότασης που είναι επίσης μια if. Η δεύτερη if ελέγχει αν το j είναι μικρότερο από το 1. Εφόσον η συνθήκη αυτή αληθεύει, τότε τυπώνει τη σειρά χαρακτήρων "i>0 και j<1".

5.1.2 Η επιλογή if…else

Η γενική μορφή της if…else έχει ως εξής:

```
if (condition) {
     statement(s);
}
else {
     statement(s);
}
```

Στην περίπτωση που η συνθήκη αληθεύει εκτελούνται οι προτάσεις μέσα στο μπλοκ της if. Αν η συνθήκη είναι ψευδής, εκτελούνται οι προτάσεις μέσα στο μπλοκ της else. Τόσο οι προτάσεις μέσα στο μπλοκ της if όσο και οι προτάσεις στο μπλοκ της else μπορεί να αρχίζουν επίσης με μια if.

```
static void ifElseDemo() { //Κώδικας 5.4
     int i = 1, j = 2;
```

```
        if (i > 0) {
            if (j < 2 && j > 0) {
                System.out.println("j ίσο με 1");
            } else if (j < 2) {
                System.out.println("j<=0");
            }
            System.out.println("i>0");
        } else {
            System.out.println("i<=0");
        }
    }
```

Κώδικας 5.4 *Παράδειγμα if…else*

Στον κώδικα 5.4 ελέγχεται καταρχάς η τιμή του i. Αν αυτή είναι θετική, η ροή συνεχίζει στο μπλοκ της εξωτερικής if, διαφορετικά η ροή διακλαδώνεται στον όρο else στο τέλος του κώδικα όπου τυπώνεται η σειρά "i<=0". Στην περίπτωση που η ροή εισέλθει στο μπλοκ της εξωτερικής if, ελέγχεται το j. Εφόσον η μεταβλητή j είναι ακέραιου τύπου και το j είναι μικρότερο του 2 και μεγαλύτερο του 0, δεν μπορεί παρά να είναι 1, οπότε τυπώνεται η σειρά "j ίσο με 1". Αν όμως η συνθήκη j < 2 && j>0 είναι ψευδής, δηλαδή το j δεν είναι 1, τότε ελέγχεται αν το j είναι μικρότερο του 2. Εφόσον το j είναι μικρότερο του 2 και δεν είναι 1, σημαίνει πως είναι μικρότερο ή ίσο του 0, οπότε τυπώνεται η σειρά "j<=0". Σε κάθε περίπτωση, μέσα στο μπλοκ της εξωτερικής if εκτυπώνεται η σειρά "i>0", ενώ στον όρο else της εξωτερικής if τυπώνεται η σειρά "i<=0".

5.1.3 Η switch

Σε μερικές περιπτώσεις ο έλεγχος της ροής του προγράμματος καθορίζεται από πολλαπλές συνθήκες. Όλες αυτές οι περιπτώσεις μπορούν να κωδικοποιηθούν με χρήση εμφωλευμένων if. Ωστόσο, ανάλογα με το πλήθος των συνθηκών, η χρήση εμφωλευμένων if αυξάνει την πολυπλοκότητα και καθιστά τον κώδικα δυσανάγνωστο. Σε κάποιες από αυτές τις περιπτώσεις ο κώδικας μπορεί να απλοποιηθεί με χρήση της switch.
Η γενική μορφή της switch έχει ως εξής:

```
switch(expression) {
     case constant1: statement(s); break;
     case constant2: statement(s); break;
     case constant3: statement(s); break;
     …
     …
     default: statement(s);
}
```

Η expression της switch πρέπει να είναι τύπου char, byte, short, int ή String. Οι τιμές ελέγχου στους όρους case πρέπει να είναι σταθερές, δηλαδή literals ή final μεταβλητές. Τα statement(s) όπως και στην if αντιπροσωπεύουν ένα ή περισσότερα statements. Ωστόσο εδώ στην περίπτωση που έχουμε περισσότερα από ένα statements δεν είναι υποχρεωτικό να τα εσωκλείσουμε σε αγκύλες. Η break ακολουθεί τα statements μίας case. Ωστόσο, η παρουσία της είναι προαιρετική ανάλογα με το αποτέλεσμα που θέλουμε να πετύχουμε. Πάντως, η break έχει ως αποτέλεσμα την έξοδο από την switch. Η default είναι επίσης προαιρετική.

Πώς δουλεύει όμως η switch; Καταρχάς αξιολογείται η expression. Ανάλογα με την τιμή της, ο έλεγχος της ροής κατευθύνεται στην κατάλληλη case. Αν σε καμία case δεν προβλέπεται η τιμή της expression, ο έλεγχος της ροής κατευθύνεται στην default εφόσον υπάρχει αλλιώς έξω από την switch. Στην περίπτωση που ταιριάζει η τιμή της expression με την τιμή ελέγχου μιας case, εκτελούνται τόσο τα

statements της αντιστοιχιζόμενης case όσο και τα statements των επόμενων case ή και της default έως ότου εκτελεστεί ένα break.

Η default δεν τοποθετείται υποχρεωτικά ως τελευταίος όρος της switch. Αντίθετα, μπορεί να τοποθετηθεί ως πρώτος όρος ή οπουδήποτε ανάμεσα στις case. Χρειάζεται όμως προσοχή ως προς το πού θα τοποθετήσουμε μια default πρόταση, γιατί σε συνδυασμό με τα break μπορεί να έχουμε εντελώς διαφορετικά αποτελέσματα.

Αποσαφηνίζουμε παρακάτω με παραδείγματα χρήσης της switch:

```
static void switchDemo() { //Κώδικας 5.5
    int cV = 2;
    final int con = 3;
    switch (cV) {
        case 1:
            System.out.print("1");
            System.out.println("-case 1");
            break;
        case 2:
            System.out.println("2");
            break;
        case con:
            System.out.println("three");
            break;
        default:
            System.out.println("default");
    }
}
```

Κώδικας 5.5 *Τυπικό παράδειγμα χρήσης της switch*

Έστω στον κώδικα 5.5, η μεταβλητή cV είναι ακέραιου τύπου. Αν η τιμή της είναι 1, η έξοδος του κώδικα θα είναι 1-case 1, δηλαδή η τιμή της μεταβλητής αντιστοιχίζεται στην case 1 και επομένως εκτελούνται οι τρεις statements της case 1. Το πρώτο statement τυπώνει 1, το δεύτερο -case 1, ενώ το τρίτο statement είναι ένα break που τερματίζει την switch. Αν η cV έχει την τιμή 2, τότε τυπώνεται το 2. Αν η τιμή της cV είναι 3, τότε τυπώνεται η σειρά "three". Προσέξτε πως αν η μεταβλητή con δεν είχε δηλωθεί ως final, ο μεταγλωττιστής θα παρήγαγε κατάλληλο λάθος.

Τέλος, αν η τιμή της cV είναι διαφορετική και από το 1 και από το 2 και από το 3, τότε εκτελείται η default και εκτυπώνεται η σειρά “default”. Σημειώστε πως η default δεν ακολουθείται από break. Πράγματι, η τοποθέτηση ενός break στον τελευταίο όρο μιας switch δεν μεταβάλλει κατά κανένα τρόπο το αποτέλεσμα της switch, καθώς μετά τον τελευταίο όρο ακολουθεί έτσι κι αλλιώς έξοδος από την switch.

```
static void switchNoBreak() { //Κώδικας 5.6
    int cV = 1;
    final int con = 3;
    switch (cV) {
        case 1:
            System.out.print("1");
        case 2:
            System.out.print("2");
        case con:
            System.out.print("3");
        default:
            System.out.println("default");
    }

}
```

Κώδικας 5.6 *Παράδειγμα switch χωρίς break*

Αν η τιμή της cV είναι 1, η έξοδος του κώδικα 5.6 είναι 123default. Αν η τιμή της είναι 2, η έξοδος του κώδικα είναι 23default. Αν η τιμή της είναι 3, η έξοδος είναι 3default. Τέλος, αν η τιμή της cV είναι διάφορη και από το 1 και από το 2 και από το 3, η έξοδος είναι default. Επομένως, αν η break απουσιάζει, τότε η switch κατευθύνεται στην κατάλληλη case και από εκτελεί όλα τα statements έως ότου βρει break ή έως ότου τερματίσει η switch.
Όπως αναφέραμε προηγουμένως, η default δεν τοποθετείται υποχρεωτικά ως τελευταίος όρος μιας switch. Ωστόσο, η συμπεριφορά της switch δεν είναι ανεξάρτητη από τη θέση της default.

```
static void switchDefault() { //Κώδικας 5.7
    int cV = 1;
    final int con = 3;
    switch (cV) {
        default:
            System.out.print("default");
        case 1:
            System.out.print("1");
        case 2:
            System.out.print("2");
        case con:
            System.out.println("3");
    }
}
```

Κώδικας 5.7 *Παράδειγμα switch με την default ως πρώτο όρο*

Αν η τιμή της cV είναι 1, η έξοδος του κώδικα 5.7 είναι 123. Αν η τιμή της είναι 2, η έξοδος του κώδικα είναι 23. Αν η τιμή της είναι 3, η έξοδος είναι 3. Τέλος, αν η τιμή της cV είναι διάφορη και από το 1 και από το 2 και από το 3, η έξοδος είναι default123.
Ας δούμε και ένα παράδειγμα όπου η expression είναι τύπου String.

```
static void switchString() { //Κώδικας 5.8
    String monthLabel = "Jan";
    String monthAA = null;
    switch (monthLabel) {
        case "Jan":
            monthAA = "first";
            break;
        case "Feb":
            monthAA = "second";
            break;
        case "Mar":
            monthAA = "third";
            break;
        case "Apr":
            monthAA = "fourth";
            break;
        case "May":
            monthAA = "fifth";
            break;
        case "Jun":
            monthAA = "sixth";
            break;
        case "Jul":
            monthAA = "seventh";
```

```
                break;
            case "Aug":
                monthAA = "eighth";
                break;
            case "Sep":
                monthAA = "ninth";
                break;
            case "Oct":
                monthAA = "tenth";
                break;
            case "Nov":
                monthAA = "eleventh";
                break;
            case "Dec":
                monthAA = "twelfth";
        }
        if (monthAA == null) {
            System.out.println("Unkown Month Name: " + monthLabel);
        } else {
            System.out.println(monthLabel + " is the " + monthAA + " month of
the year");
        }
    }
```

Κώδικας 5.8 *Παράδειγμα switch με έκφραση ελέγχου τύπου String*

5.1.4 Ο τριαδικός τελεστής

Ο τριαδικός τελεστής που είδαμε ήδη στην ενότητα 4.1.5.3, αποτελεί μια συνοπτική δομή επιλογής. Παρουσιάζει μεγάλη ομοιότητα με τη δομή επιλογής if…else. Για παράδειγμα, ο κώδικας 5.9, επιτυγχάνει δύο φορές το ίδιο αποτέλεσμα, την πρώτη χρησιμοποιεί τον τριαδικό τελεστή και τη δεύτερη την δομή if…else.

```
    static void ternaryOp() { //Κώδικας 5.9
        int i = 1, j = 2, max;

        max = i > j ? i : j;

        if (i > j) {
            max = i;
        } else {
            max = j;
        }
    }
```

Κώδικας 5.9 *Παράδειγμα αναλογίας τριαδικού τελεστή και if…else*

5.2 Επαναληπτικές δομές

Οι επαναληπτικές δομές αποτελούν παρόμοιες με τις δομές επιλογής θεμελιώδεις δομές κάθε γλώσσας προγραμματισμού. Αναλογιστείτε πως χωρίς αυτές αν επιθυμούσαμε να τυπώσουμε τους ακέραιους από το 1 έως το 100.000, θα έπρεπε να πληκτρολογήσουμε 100.000 εντολές. Στην ουσία, ο αποτελεσματικός προγραμματισμός των Ηλεκτρονικών Υπολογιστών θα ήταν αδύνατος. Στα αγγλικά, η επαναληπτική δομή αναφέρεται με τον όρο loop που αποδίδεται στα ελληνικά ως βρόχος. Οι επαναληπτικές δομές που υποστηρίζει η Java είναι η while, η do-while και η for.

5.2.1 Η επαναληπτική δομή while

Η γενική μορφή της επαναληπτικής δομής while έχει ως εξής:

```
while(condition) {
     statement(s);
}
```

Η while εξετάζει τη συνθήκη condition και όσο αυτή είναι αληθής εκτελεί μια ή περισσότερες statements. Παρόμοια με την if, αν έχουμε μια μόνο statement, οι αγκύλες είναι προαιρετικές.

```
static void whileDemo() { //Κώδικας 5.10
    int i = 10;
    while (i > 0) {
        System.out.print("i>0 ");
        i--;
    }
    System.out.println("i=" + i);
}
```

Κώδικας 5.10 *Παράδειγμα απλής while*

Στον κώδικα 5.10 η ακέραιη μεταβλητή i αρχικοποιείται στην τιμή 10. Στη συνέχεια η while εξετάζει αν το i είναι μεγαλύτερο από το 0. Μέσα στο μπλοκ της while τυπώνεται το μήνυμα "i>0" και ακολούθως μειώνεται η τιμή του i κατά 1. Η διαδικασία επαναλαμβάνεται έως ότου το i πάψει να είναι μεγαλύτερο από το 0. Δεδομένου ότι το i αρχικοποιήθηκε στο 10 και η τιμή του μειώνεται κατά 1 σε κάθε επαναληπτικό βήμα, το i θα γίνει 0 ακριβώς μετά από 10 επαναλήψεις. Επομένως, ο κώδικας 5.10, τυπώνει 10 φορές τη σειρά "i>0", στη συνέχεια το i γίνεται 0, η συνθήκη i>0 καθίσταται ψευδής και ο κώδικας εξέρχεται από τον βρόχο while, οπότε εκτελεί την επόμενη πρόταση και τυπώνει i=0.

Προσέξτε πως αν στον κώδικα 5.10 η μεταβλητή i είχε αρχικοποιηθεί σε τιμή μικρότερη ή ίση με το 0, η συνθήκη ελέγχου θα ήταν εξαρχής ψευδής και ο κώδικας δεν θα εισερχόταν ποτέ στο μπλοκ της while. Αντίθετα, αν η μεταβλητή i δεν μειωνόταν μέσα στην while, τότε η ροή του κώδικα δεν θα εξερχόταν από τον βρόχο. Σε αυτήν την περίπτωση λέμε ότι έχουμε έναν ατέρμονο βρόχο (endless loop). Προφανώς, η παρουσία ατέρμονου βρόχου μέσα σε ένα πρόγραμμα αποτελεί σοβαρό προγραμματιστικό λάθος.

Τόσο η while όσο και οι υπόλοιπες επαναληπτικές δομές μπορούν να αξιοποιήσουν τις λέξεις-κλειδιά break και continue. Η μεν break μεταφέρει τη ροή του προγράμματος στην πρώτη πρόταση έξω από το μπλοκ της while, η δε continue μεταφέρει τη ροή στη συνθήκη ελέγχου.

Ας υποθέσουμε πως θέλουμε να τυπώσουμε 10 το πολύ τυχαίες ακέραιες τιμές στην κλίμακα 1 έως 10. Αν όμως κάποια από τις τιμές αυτές είναι μικρότερη από το 2, τότε να την τυπώσουμε και να σταματήσουμε ακόμη και αν δεν έχει συμπληρωθεί ο αριθμός των 10 εκτυπωμένων τιμών.
Ένας τρόπος για να πετύχουμε αυτήν την απαίτηση δίνεται στον κώδικα 5.11.

```
static void prtRandoms() {
    int i = 0, rI;
    Random generator = new Random();
    while (i < 10) {
        i++;
        rI = generator.nextInt(10) + 1;
        if (rI >= 2) {
            System.out.println(rI);
        } else {
            System.out.println(rI);
            break;
        }
```

```
        }
    }
```

Κώδικας 5.11 *Παράδειγμα while με break*

Καταρχάς στον κώδικα 5.11 δηλώνουμε και αρχικοποιούμε τη μεταβλητή i στο 0. Παράλληλα δηλώνουμε τη μεταβλητή rI. Στη συνέχεια δημιουργούμε το αντικείμενο generator. Πρόκειται για μια γεννήτρια τυχαίων αριθμών. Σημειώστε πως για να χρησιμοποιήσετε ένα τέτοιο αντικείμενο θα πρέπει να εισάγετε στο πρόγραμμά σας τη βιβλιοθήκη Java.util.Random. Αυτό επιτυγχάνεται αν εισάγετε αμέσως μετά τη δήλωση του πακέτου τη σειρά import Java.util.Random; Στη συνέχεια αναπτύσσουμε μια επαναληπτική διαδικασία while. Η συνθήκη ελέγχου είναι i<10, δηλαδή η επαναληπτική διαδικασία θα επαναλαμβάνεται όσο το i θα είναι μικρότερο από το 10. Δεδομένου ότι το i αρχικοποιήθηκε στο 0 και σε κάθε επανάληψη αυξάνεται κατά 1, θα εκτελεστούν 10 το πολύ επαναλήψεις. Αυτό είναι συμβατό με την απαίτηση να τυπώσουμε 10 το πολύ τυχαίους ακέραιους. Στη συνέχεια χρησιμοποιούμε τον generator για να παράξουμε έναν τυχαίο αριθμό. Η συνάρτηση nextInt παράγει έναν τυχαίο ακέραιο από το 0 έως την τιμή του ορίσματός της μείον 1, δηλαδή από το 0 έως το 9. Σε αυτόν τον αριθμό προσθέτουμε το 1, οπότε ο ακέραιος που εκχωρείται στη μεταβλητή rI είναι ένας τυχαίος από το 1 έως το 10. Μετά ελέγχουμε την τιμή της rI και αν είναι μεγαλύτερη ή ίση με το 2, απλώς την τυπώνουμε διαφορετικά, ισχύει η απαίτηση σύμφωνα με την οποία αν κάποια από τις τυχαίες τιμές είναι μικρότερη από το 2, τότε θα πρέπει να την τυπώσουμε και να σταματήσουμε περαιτέρω εκτύπωση τυχαίων αριθμών. Αυτό ακριβώς κάνει ο κώδικας 5.11. Στη συνέχεια τυπώνουμε την τιμή που είναι μικρότερη από το 2 και αμέσως μετά εκτελείται η break που διακόπτει την επαναληπτική διαδικασία και μεταφέρει τη ροή έξω από την επαναληπτική διαδικασία όπου ακολουθεί και η έξοδος της συνάρτησης prtRandoms.
Δύο σχόλια σχετικά με τον κώδικα 5.11. Γενικά είναι καλό στα προγράμματά μας να αποφεύγουμε τη φλυαρία, δηλαδή όπου είναι εφικτό να προτιμούμε τον βραχύτερο έναντι του μακρύτερου κώδικα. Προσέξτε πως ο κώδικας System.out.println(rI) εκτελείται και στις δύο διακλαδώσεις της if, δηλαδή είτε ισχύει η συνθήκη rI>=2 είτε όχι. Ένας τέτοιος κώδικας μπορεί να βγει πάντα έξω από την if έτσι ώστε μια παρουσία του να είναι αρκετή.

Ένα άλλο θέμα που θα μπορούσαμε να βελτιώσουμε σχετίζεται με την αύξηση του i. Αφού ελέγξουμε το i στην επόμενη γραμμή, το αυξάνουμε κατά 1. Το ίδιο ακριβώς αποτέλεσμα μπορούμε να πετύχουμε αν αυξήσουμε το i στη συνθήκη ελέγχου με χρήση του μεταθεματικού τελεστή προσαύξησης. Προσοχή όμως! Αν η αύξηση γίνει όπως γίνεται στον κώδικα 5.11, δεν έχει ιδιαίτερη σημασία αν χρησιμοποιηθεί ο προθεματικός ή ο μεταθεματικός τελεστής προσαύξησης. Αν όμως η αύξηση γίνει μέσα στη συνθήκη, τότε η χρήση του μεταθεματικού τελεστή διαφοροποιεί ουσιαστικά τη συμπεριφορά του κώδικα. Μελετήστε πώς την αλλάζει και γιατί. Ο κώδικας 5.12 παράγει το ίδιο ακριβώς αποτέλεσμα με τον κώδικα 5.11 με το πλεονέκτημα ότι είναι βραχύτερος.

```
static void prtRandoms() { //Κώδικας 5.12
    int i = 0, rI;
    Random generator = new Random();
    while (i++ < 10) {
        rI = generator.nextInt(10) + 1;
        System.out.println(rI);
        if (rI < 2) {
            break;
        }
    }
}
```

Κώδικας 5.12 *Βραχύτερη έκδοση της συνάρτησης prtRandoms*

Ας υποθέσουμε τώρα πως θέλουμε να τυπώσουμε 10 τυχαίες ακέραιες τιμές στην κλίμακα 1 έως 10 εξαιρώντας τις τιμές 3 και 4, δηλαδή 10 τιμές που ανήκουν στο σύνολο {1..10}-{3,4}. Η λέξη-κλειδί continue θα μας βοηθήσει στην επίλυση αυτού του προβλήματος.